

https://docs.google.com/presentation/d/1hY1lcNvcWUMfN3an7kn0-hnaFx-dfLqyslW2nz7BEbQ/edit#slide=id.g51da6455d5_0_0

More Rust

Course Leads:

Sara Magaziotis-Ginori

Olivia Li

Isaac Oh

Aman Shah

Elson Liu

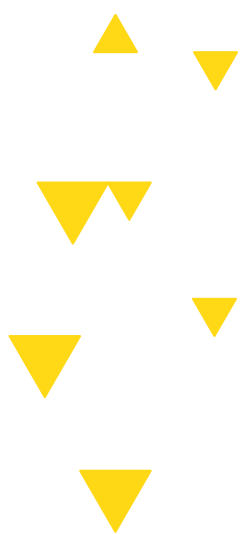
Fujia Wang



BLOCKCHAIN
AT BERKELEY



O Announcements





Announcements

Moves

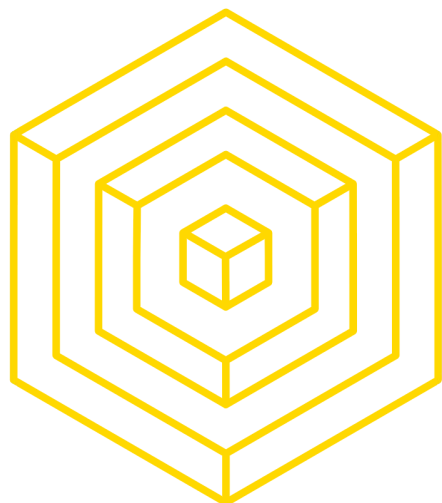
- Homework #7 was released this Monday
 - Due next Wednesday (11/5)
- Email dev-decal@blockchain.berkeley.edu for extensions

Source: <https://google.github.io/comprehensive-rust/ownership/moved-strings-rust.html>

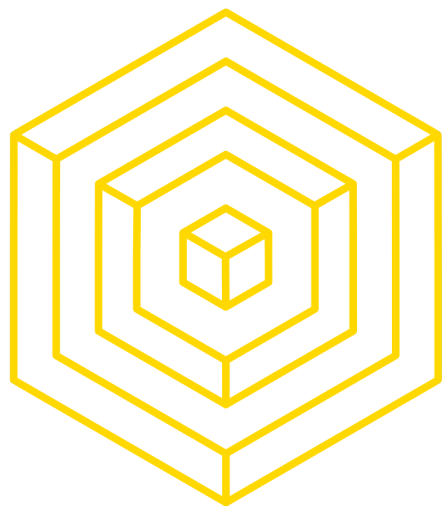


LECTURE OUTLINE

- 1 More about ownership
- 2 Other Quirks
- 3 Cosm/Wasm
- 4 Cargo



1 More ownership



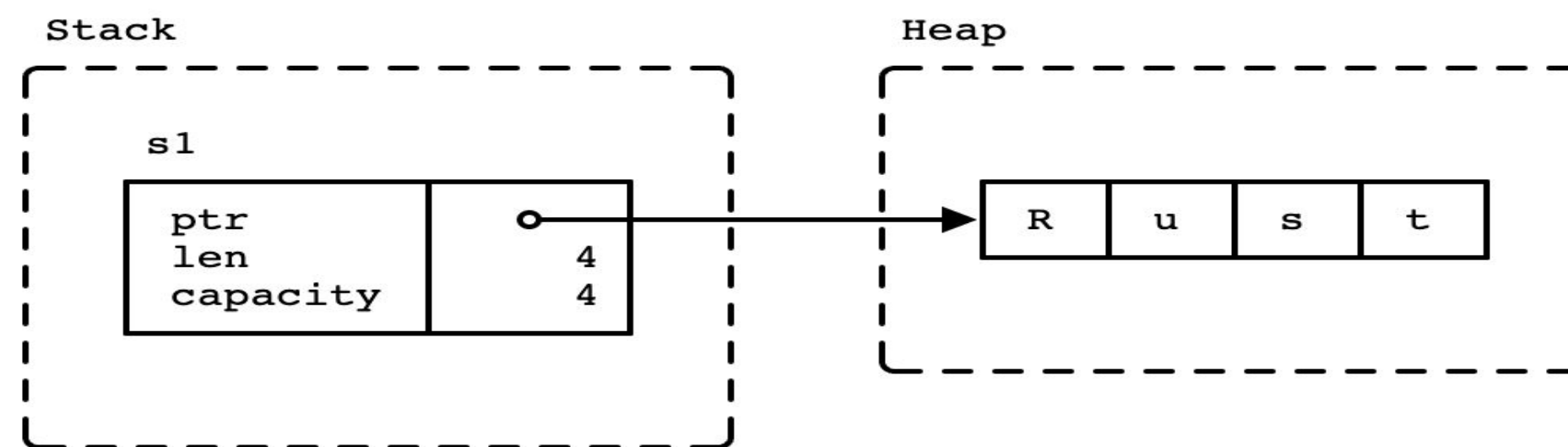
More ownership

Moves

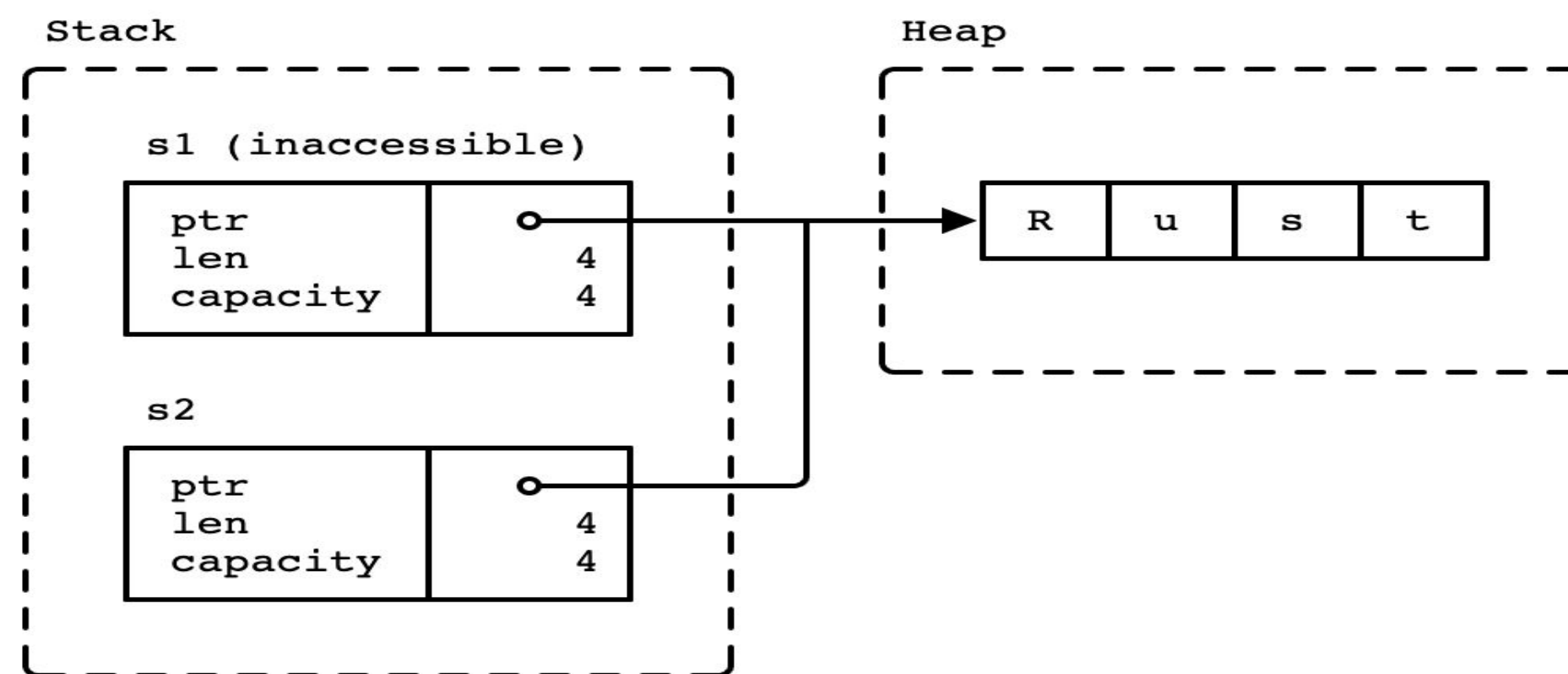
```
1 fn main() {
2     let s1: String = String::from("Rust");
3     let s2: String = s1;
4 }
```

- The heap data from `s1` is reused for `s2`.
- When `s1` goes out of scope, nothing happens (it has been moved from).

Before move to `s2`:



After move to `s2`:





More ownership

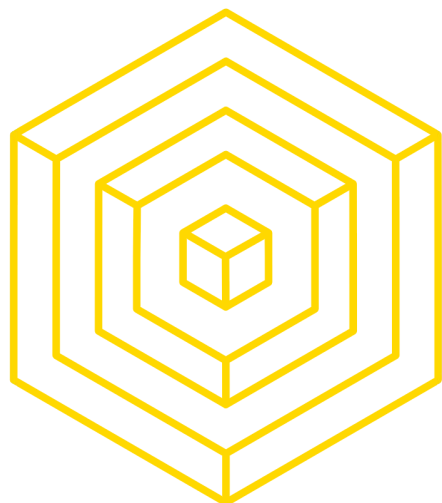
Functions and ownership

When you pass a value to a function, the value is assigned to the function parameter. This transfers ownership:

```
1 fn say_hello(name: String) {  
2     println!("Hello {name}")  
3 }  
4  
5 fn main() {  
6     let name = String::from("Alice");  
7     say_hello(name);  
8     // say_hello(name);  
9 }
```

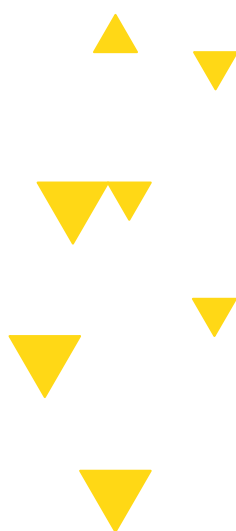
▼ Speaker Notes

- With the first call to `say_hello`, `main` gives up ownership of `name`. Afterwards, `name` cannot be used anymore within `main`.
- The heap memory allocated for `name` will be freed at the end of the `say_hello` function.
- `main` can retain ownership if it passes `name` as a reference (`&name`) and if `say_hello` accepts a reference as a parameter.
- Alternatively, `main` can pass a clone of `name` in the first call (`name.clone()`).
- Rust makes it harder than C++ to inadvertently create copies by making move semantics the default, and by forcing programmers to make clones explicit.



1.5

How to Solve Membership Issues





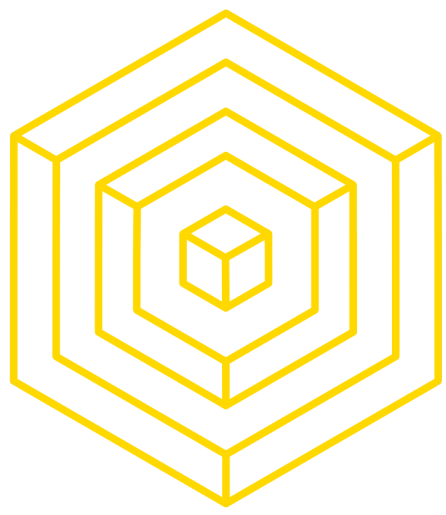
How to Solve Membership Issues

Borrow immutably

Borrow immutably: &T

- Many readers allowed at the same time.
- Callee cannot mutate the value.

```
fn show_len(s: &String) { println!("{}", s.len()); }  
  
let s = String::from("hello");  
show_len(&s);           // borrow  
println!("{}", s);      // still valid
```



How to Solve Membership Issues

Borrow immutably - Example

Instead of transferring ownership when calling a function, you can let a function *borrow* the value:

```
1  #[derive(Debug)]
2  struct Point(i32, i32);
3
4  fn add(p1: &Point, p2: &Point) -> Point {
5      Point(p1.0 + p2.0, p1.1 + p2.1)
6  }
7
8  fn main() {
9      let p1 = Point(3, 4);
10     let p2 = Point(10, 20);
11     let p3 = add(&p1, &p2);
12     println!("{p1:?} + {p2:?} = {p3:?}");
13 }
```

- The `add` function *borrow*s two points and returns a new point.
- The caller retains ownership of the inputs.

Source: <https://google.github.io/comprehensive-rust/ownership/moved-strings-rust.html>



How to Solve Membership Issues

Borrow mutably

Borrow mutably: &mut T

- Exactly one mutable borrower at a time, and no other borrows while it is active.
- Callee may modify the value

```
fn push_excl(s: &mut String) { s.push('!'); }

let mut s = String::from("hi");
push_excl(&mut s);           // exclusive mutable borrow
println!("{}", s);           // "hi!"
```



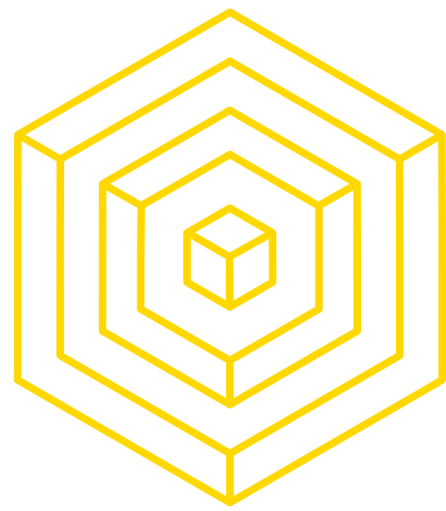

How to Solve Membership Issues

Pass by value (move)

Pass by value (move): T

- Ownership transfers to the callee. The caller cannot use the variable afterward unless ownership is returned.

```
fn take(s: String) { /* owns s */ }  
  
let s = String::from("x");  
take(s);      // moved  
// s is now invalid
```



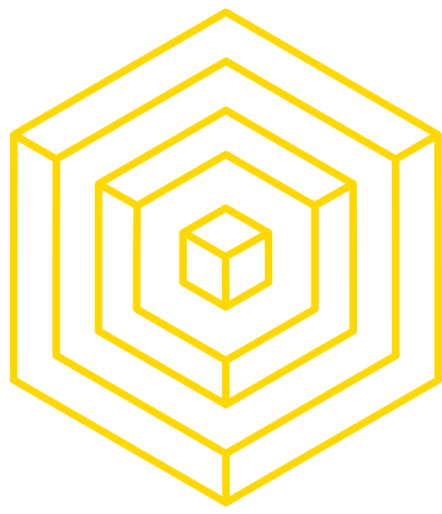
How to Solve Membership Issues

Clone when you truly need another owner

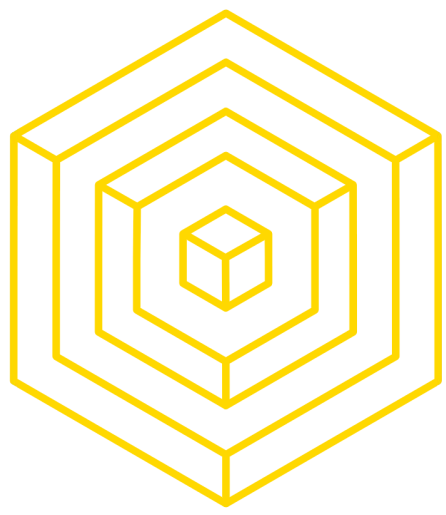
Clone when you truly need another owner: T: Clone

- Makes a deep copy of heap data.
- Use sparingly; prefer borrowing when possible.

```
fn store(s: String) { /* keeps it */ }  
  
let s = String::from("data");  
store(s.clone());           // caller keeps its own copy  
println!("{}", s);
```



2 Other quirks



Quirks

Control flow

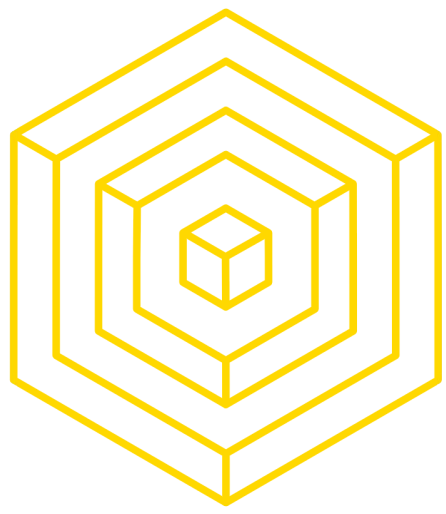
15

The `if let expression` lets you execute different code depending on whether a value matches a pattern:

```
1 fn main() {  
2     let arg = std::env::args().next();  
3     if let Some(value) = arg {  
4         println!("Program name: {value}");  
5     } else {  
6         println!("Missing name?");  
7     }  
8 }
```

See [pattern matching](https://google.github.io/comprehensive-rust/control-flow/novel.html) for more details on patterns in Rust.

Source: <https://google.github.io/comprehensive-rust/control-flow/novel.html>



Quirks

Control flow

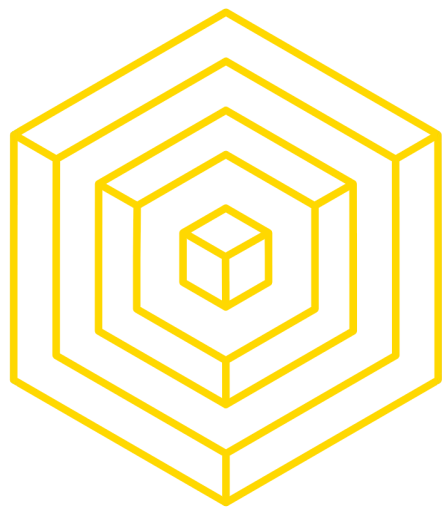
16

Like with `if let`, there is a `while let` variant which repeatedly tests a value against a pattern:

```
1 fn main() {  
2     let v = vec![10, 20, 30];  
3     let mut iter = v.into_iter();  
4  
5     while let Some(x) = iter.next() {  
6         println!("x: {x}");  
7     }  
8 }
```

Here the iterator returned by `v.into_iter()` will return a `Option<i32>` on every call to `next()`. It returns `Some(x)` until it is done, after which it will return `None`. The `while let` lets us keep iterating through all items.

Source: <https://google.github.io/comprehensive-rust/control-flow/novel.html>



Quirks

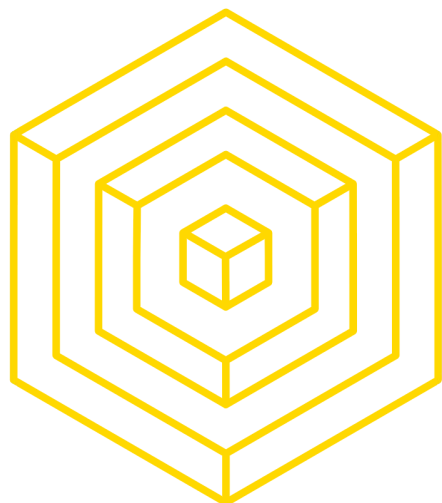
Control flow

The `match` keyword is used to match a value against one or more patterns. In that sense, it works like a series of `if let` expressions:

```
1 fn main() {  
2     match std::env::args().next().as_deref() {  
3         Some("cat") => println!("Will do cat things"),  
4         Some("ls")  => println!("Will ls some files"),  
5         Some("mv")  => println!("Let's move some files"),  
6         Some("rm")  => println!("Uh, dangerous!"),  
7         None       => println!("Hmm, no program name?"),  
8         _          => println!("Unknown program name!"),  
9     }  
10 }
```

Like `if let`, each match arm must have the same type. The type is the last expression of the block, if any. In the example above, the type is `()`.

Source: <https://google.github.io/comprehensive-rust/control-flow/novel.html>

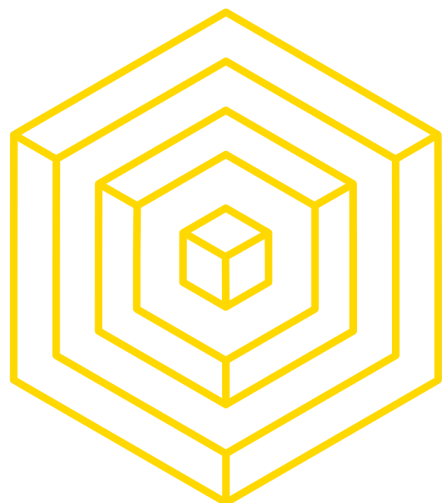


Quirks

Control flow

If you already have variables with the right names, then you can create the struct using a shorthand:

```
1  #[derive(Debug)]
2  struct Person {
3      name: String,
4      age: u8,
5  }
6
7  impl Person {
8      fn new(name: String, age: u8) -> Person {
9          Person { name, age }
10     }
11 }
12
13 fn main() {
14     let peter = Person::new(String::from("Peter"), 27);
15     println!("{peter:?}");
16 }
```

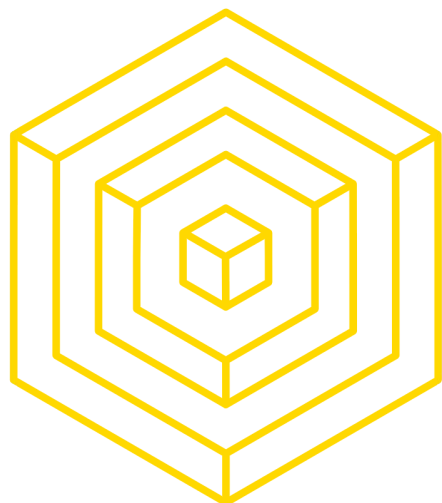


Quirks

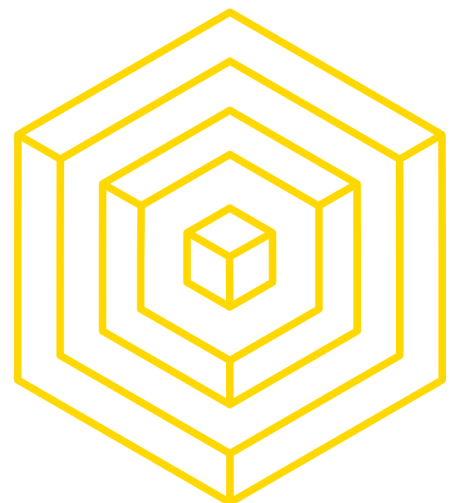
Control flow

Like C and C++, Rust has support for custom structs:

```
1 struct Person {  
2     name: String,  
3     age: u8,  
4 }  
5  
6 fn main() {  
7     let mut peter = Person {  
8         name: String::from("Peter"),  
9         age: 27,  
10    };  
11    println!("{}", peter.name, peter.age);  
12  
13    peter.age = 28;  
14    println!("{}", peter.name, peter.age);  
15  
16    let jackie = Person {  
17        name: String::from("Jackie"),  
18        ..peter  
19    };  
20    println!("{}", jackie.name, jackie.age);  
21 }
```



3 Cosm/Wasm



Cosm/Wasm

WebAssembly is a low-level assembly-like language with a compact binary format that runs with near-native performance and provides languages such as C/C++ and Rust, and many more with a compilation target so that they run on the web. Support for memory-managed languages such as Java, Kotlin, and Dart is in the works and should become available soon. WebAssembly is designed to run alongside JavaScript, allowing both to work together.

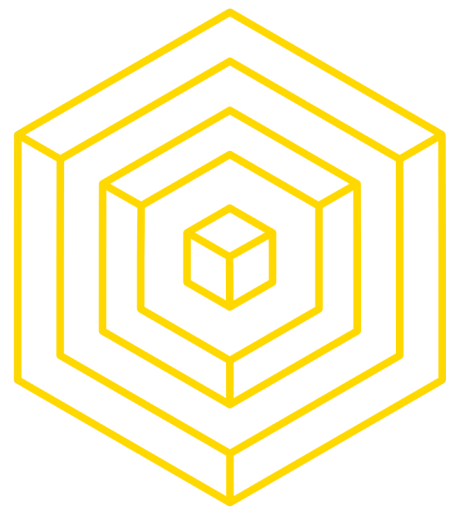
Source: <https://google.github.io/comprehensive-rust/control-flow/novel.html>



Cosm/Wasm

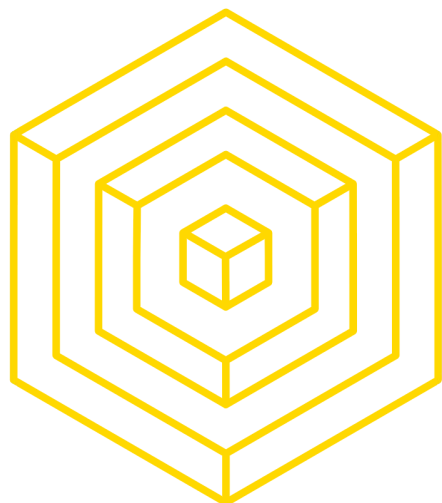
WebAssembly code (binary code, that is, bytecode) is intended to be run on a portable virtual stack machine (VM). The bytecode is designed to be faster to parse and execute than JavaScript and to have a compact code representation.

Source: <https://web.dev/articles/what-is-webassembly>



Cosm/Wasm

- Javascript can be compiled down to WASM
- Better performance
- Rust can also be compiled down to WASM



4 Cargo



Cargo

Installation Steps

25

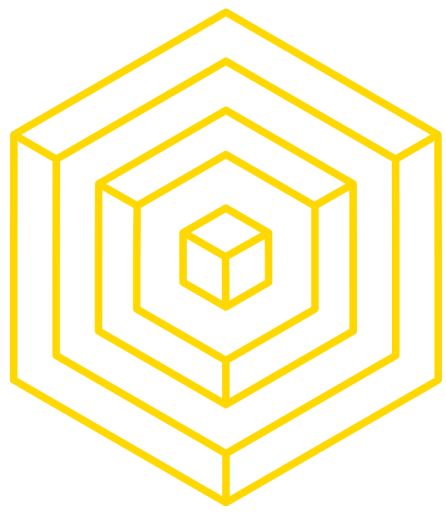
The easiest way to get Cargo is to install the current stable release of [Rust](#) by using [rustup](#). Installing Rust using `rustup` will also install `cargo`.

On Linux and macOS systems, this is done as follows:

```
curl https://sh.rustup.rs -sSf | sh
```

It will download a script, and start the installation. If everything goes well, you'll see this appear:

```
Rust is installed now. Great!
```



Cargo

Crates

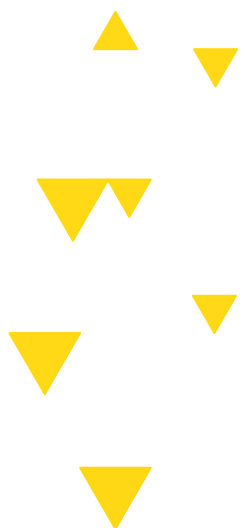
Crate

A Rust *crate* is either a library or an executable program, referred to as either a *library crate* or a *binary crate*, respectively.

Every [target](#) defined for a Cargo [package](#) is a *crate*.

Loosely, the term *crate* may refer to either the source code of the target or to the compiled artifact that the target produces. It may also refer to a compressed package fetched from a [registry](#).

The source code for a given crate may be subdivided into [modules](#).





Specifying dependencies from git repositories

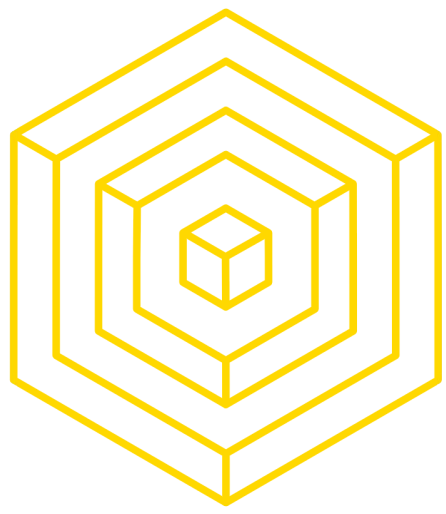
To depend on a library located in a `git` repository, the minimum information you need to specify is the location of the repository with the `git` key:

```
[dependencies]
regex = { git = "https://github.com/rust-lang/regex.git" }
```



Cargo will fetch the `git` repository at this location then look for a `Cargo.toml` for the requested crate anywhere inside the `git` repository (not necessarily at the root — for example, specifying a member crate name of a workspace and setting `git` to the repository containing the workspace).

Since we haven't specified any other information, Cargo assumes that we intend to use the latest commit on the default branch to build our package, which may not necessarily be the main branch. You can combine the `git` key with the `rev`, `tag`, or `branch` keys to specify something else. Here's an example of specifying that you want to use the latest commit on a branch named `next`:

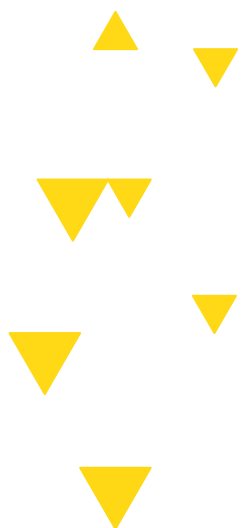


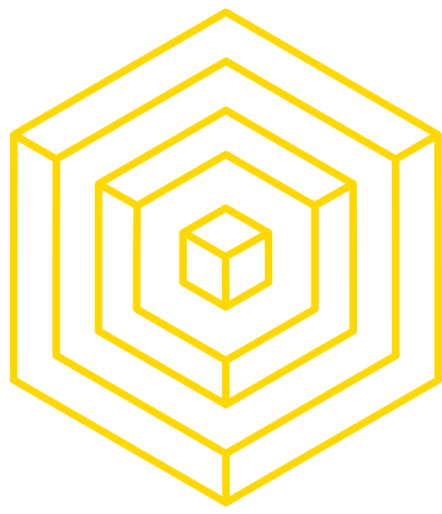
Cargo

This command manages Cargo's local set of installed binary crates. Only packages which have executable `[[bin]]` or `[[example]]` targets can be installed, and all executables are installed into the installation root's `bin` folder. By default only binaries, not examples, are installed.

The installation root is determined, in order of precedence:

- `--root` option
- `CARGO_INSTALL_ROOT` environment variable
- `install.root` Cargo [config value](#)
- `CARGO_HOME` environment variable
- `$HOME/.cargo`





5 Review



Review

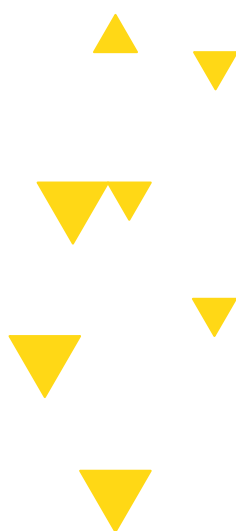
30

- Talk with a partner!!
- **What are the big takeaways from the class so far?**



Review

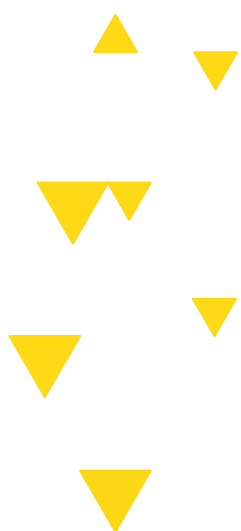
- Talk with a partner!!
- What are the big takeaways from the class so far?
- **What is your interest in blockchain/why are you motivated to take this class?**





Review

- Talk with a partner!!
- What are the big takeaways from the class so far?
- What is your interest in blockchain/why are you motivated to take this class?
- **What makes Rust different from other programming languages?**





Attendance/Vitamin

33

<https://forms.gle/NJ8aorwiK5FpD7vz7>

